

Why Average Minibatch Loss Can Increase in SGD

What the reported epoch loss measures, and why it need not fall monotonically

Paul Agron

September 29, 2026

1 The apparent contradiction

When gradient descent is introduced, we are taught that moving in the direction of the negative gradient should reduce the loss. It is therefore natural to be puzzled when, during neural-network training with stochastic gradient descent (SGD), the reported training loss occasionally rises from one epoch to the next even though the learning rate is reasonable and the optimizer is behaving normally.

The key is that the loss commonly reported for an epoch is not the full-dataset loss evaluated once at one fixed set of weights. In minibatch SGD, the weights change after every minibatch, so the epoch loss is an average of losses measured on different subsets of the data at different points in parameter space. That quantity has no requirement to decrease monotonically.

2 Minibatch SGD over an epoch

Suppose the training set contains examples $(\mathbf{x}_i, y_i)$, for $i = 1, \dots, N$, and let $\mathbf{w}$ denote the complete vector of trainable parameters of the network. Define the loss for a single training example as

$$\ell_i(\mathbf{w}) = \ell(f_{\mathbf{w}}(\mathbf{x}_i), y_i), \quad (1)$$

where $f_{\mathbf{w}}(\mathbf{x}_i)$ is the network prediction and $\ell(\cdot, \cdot)$ is the chosen loss function, such as cross-entropy or squared error.

At the beginning of an epoch, the training examples are typically shuffled and partitioned without replacement into M disjoint minibatches $B_1, B_2, \dots, B_M$. Thus every training example appears exactly once during the epoch. For a fixed parameter vector $\mathbf{w}$, the loss on minibatch B_k is

$$L_{B_k}(\mathbf{w}) = \frac{1}{|B_k|} \sum_{i \in B_k} \ell_i(\mathbf{w}). \quad (2)$$

If all minibatches were evaluated using the same fixed $\mathbf{w}$, then, assuming equal batch sizes,

$$\frac{1}{M} \sum_{k=1}^M L_{B_k}(\mathbf{w}) = L_{\text{full}}(\mathbf{w}), \quad \text{where} \quad L_{\text{full}}(\mathbf{w}) = \frac{1}{N} \sum_{i=1}^N \ell_i(\mathbf{w}). \quad (3)$$

So, with fixed weights, the average minibatch loss does equal the full-dataset loss.

3 The weights change between minibatches

During training, however, the weights are not fixed. After minibatch B_k , SGD performs the update

$$\mathbf{w}_{k+1} = \mathbf{w}_k - \eta \nabla L_{B_k}(\mathbf{w}_k), \quad (4)$$

where η is the learning rate. Therefore, the losses observed during the epoch are actually

$$L_{B_1}(\mathbf{w}_1), \quad L_{B_2}(\mathbf{w}_2), \quad \dots, \quad L_{B_M}(\mathbf{w}_M),$$

and the reported average training loss for the epoch is approximately

$$\bar{L}_{\text{epoch}} = \frac{1}{M} \sum_{k=1}^M L_{B_k}(\mathbf{w}_k). \quad (5)$$

The reported quantity. Unlike eq. (3), each term in eq. (5) is evaluated at a different parameter vector. In general, therefore,

$$\bar{L}_{\text{epoch}} \neq L_{\text{full}}(\mathbf{w}_{\text{end}}) :$$

the epoch loss is not the full-dataset loss at the final weights, nor at any single set of weights.

4 Why one epoch's average can exceed the previous one

Each minibatch gradient is based only on a subset of the training data, so in general

$$\nabla L_{B_k}(\mathbf{w}_k) \neq \nabla L_{\text{full}}(\mathbf{w}_k). \quad (6)$$

An update that reduces the current minibatch loss can increase the loss of other training examples. After reshuffling, the next epoch also uses different minibatch compositions and a different ordering, so the optimizer follows a different sequence of approximate gradients through parameter space.

Consequently, there is no requirement that $\bar{L}_{\text{epoch}}^{(t+1)} < \bar{L}_{\text{epoch}}^{(t)}$ for successive epochs t and $t+1$. A training history such as

$$0.42, \quad 0.39, \quad 0.40, \quad 0.36$$

is therefore fully compatible with normal minibatch SGD. The temporary increase from 0.39 to 0.40 does not by itself imply that the learning rate is too large or that gradient descent is oscillating around a minimum.

5 What the descent guarantee actually applies to

For full-batch gradient descent, a sufficiently small step taken along the exact negative gradient has a local descent guarantee because the update uses $\nabla L_{\text{full}}(\mathbf{w})$. Minibatch SGD instead uses $\nabla L_{B_k}(\mathbf{w})$, which is only an estimate of the full gradient. Moreover, the commonly reported epoch loss is an average of minibatch losses evaluated at changing weights rather than the value of one fixed objective at one fixed point.

Conclusion. Even with a sensible learning rate and no momentum, the average minibatch loss over successive epochs need not decrease monotonically. Small upward movements in the epoch-average training loss can be a normal consequence of minibatch SGD rather than evidence of a failed optimization step.